

Clean and Prepare Data for Analysis with Python

Study Guide for Scientific and Time Series Data

This guide explains how to turn imperfect observations into an analysis-ready dataset with NumPy and pandas. The accompanying Python script runs a complete workflow, while the Jupyter notebook provides guided practice and solutions. The central example is a light curve with slow drift, transit-like dips, isolated spikes, uneven sampling, missing values, and measurement noise.

Estimated study time: 60–90 minutes. Use the Python environment at `/Users/victorbui/venvs/ai312`. Keep the script and notebook in the same folder. Run the script with:

```
# Execute the complete course and create the output tables and figures.
/Users/victorbui/venvs/ai312/bin/python \
    Python_Clean_Prepare_Data_Course.py
```

Contents

1	Learning plan	2
2	Inspect data before cleaning	2
2.1	A useful inspection checklist	3
3	Detect and handle outliers	3
4	Work with missing data	4
5	Interpolate missing values	4
6	Shift and scale data	5
6.1	Centering	5
6.2	Z-scores	5
6.3	Relative and range scaling	6
7	Remove unwanted trends	6
8	Smooth noisy data	7
9	Build a reviewable pipeline	7

9.1 Suggested output schema	8
10 Practice questions	8
11 Quick reference	9

1 Learning plan

The workflow follows the order in which decisions should normally be made. Inspect the raw records before applying a method. Standardize missing-value codes before calculating statistics. Detect unusual observations in context. Interpolate only gaps that the data can support. Apply scaling, detrending, and smoothing with a stated analytical purpose. Finish with explicit checks and an audit table.

Stage	Main question	Evidence to retain
Inspection	What does each row and column mean?	Shape, types, units, ranges, time order
Missing data	Which markers mean unavailable?	Counts, locations, gap lengths, chosen policy
Outliers	Which values violate the local pattern?	Method, window, threshold, flagged rows
Interpolation	Which gaps support estimation?	Sample locations, method, maximum gap
Normalization	Which comparison needs a common scale?	Fitted mean, spread, range, units
Detrending	Which baseline obscures the event?	Model degree, excluded events, residuals
Smoothing	Which variation is noise?	Window width, direction, statistic
Validation	Did the workflow preserve meaning?	Assertions, row counts, output schema

Key distinction. Missing values indicate unavailable measurements. Outliers are observed values that appear inconsistent with an expected pattern. Noise is short-scale variation. A trend is structured change over a longer scale. These conditions require different evidence and should not share one automatic treatment.

2 Inspect data before cleaning

Start by confirming what the table represents. For time series data, verify that timestamps are ordered and determine whether the spacing is regular. Read units and instrument documentation. Plot the data because summaries can hide spikes, gaps, and drift.

```
# Inspect structure, missingness, order, and numeric ranges.
print(data.shape)
print(data.dtypes)
print(data.isna().sum())
print(data.describe())
assert data["time_days"].is_monotonic_increasing
```

```
ax = data.plot(x="time_days", y="brightness", figsize=(10, 4))
ax.set(xlabel="Time (days)", ylabel="Relative brightness")
```

A sentinel such as -999 may represent a missing measurement. Until it is replaced, the minimum, mean, standard deviation, trend fit, and plots treat it as a valid number. Only replace a code when metadata or domain knowledge defines its meaning.

```
# Convert documented codes to the standard numeric missing value.
data["brightness"] = data["brightness"].replace(-999, np.nan)
```

2.1 A useful inspection checklist

- Confirm the unit and meaning of every variable used in the analysis.
- Check duplicate rows and duplicate timestamps separately.
- Count explicit missing values and documented sentinel codes.
- Look for impossible values using domain limits.
- Inspect time order, sample spacing, and gap length.
- Compare a plot with numeric summaries before changing the data.

3 Detect and handle outliers

An outlier is unusual relative to a specified reference. A global mean and standard deviation may work for stable, symmetric data, but they are sensitive to extreme observations. Quartile rules are more robust. A local signal often needs a moving reference because a slow trend changes its center over time.

The course uses a rolling median and median absolute deviation (MAD). Within a local window, let m be the median. The raw MAD is

$$\text{MAD} = \text{median}(|x_i - m|).$$

For nearly normal noise, 1.4826 MAD estimates the standard deviation. A point is flagged when its absolute deviation exceeds a chosen multiple of that robust scale.

```
# Estimate a local center and robust local scale.
center = values.rolling(41, center=True, min_periods=9).median()
deviation = (values - center).abs()
mad = deviation.rolling(41, center=True, min_periods=9).median()
robust_scale = 1.4826 * mad
outlier = (deviation > 5 * robust_scale) & values.notna()

# Preserve the timestamp and mark the suspect measurement as missing.
cleaned = data.copy()
cleaned.loc[outlier, "brightness"] = np.nan
```

The window should cover enough ordinary variation to estimate a stable local center without spanning unrelated behavior. The threshold controls sensitivity: a smaller number flags more observations. Review

flagged points on the raw plot. A scientifically meaningful event can be rare and extreme without being an error.

Removal versus marking. Dropping an outlier row also removes its time coordinate. Marking its measurement as missing retains the sampling grid and records where information became unavailable. This often makes later gap logic clearer.

4 Work with missing data

There are three broad strategies. Ignoring missing values during a reduction retains the measured observations. Removing rows creates a complete subset but can discard useful values from other columns. Replacing values preserves shape and alignment but adds estimates that were not observed.

Strategy	Appropriate use	Main limitation
Ignore in a calculation	A reduction can use available values and the missing mechanism does not invalidate it	Different calculations may handle missing values differently
Remove rows	Missing rows are few and removal does not distort the sample or break alignment	Valid values in other columns may be lost
Replace with a constant or statistic	The replacement has a documented meaning, or a simple baseline is explicitly required	Variability and relationships can be distorted
Interpolate	Nearby observations support an estimate within a short gap	Filled values can look more certain than they are
Model-based imputation	A justified model uses other information and uncertainty is tracked	Results depend on model assumptions

```
# Locate, count, ignore, remove, or replace missing values.
missing = data["brightness"].isna()
count = int(missing.sum())
mean_available = data["brightness"].mean()
complete_rows = data.dropna(subset=["brightness"])
median_filled = data["brightness"].fillna(data["brightness"].median())
```

When a plotted line contains NaN, it breaks at the gap. Dropping those rows can connect the two measured segments and visually imply continuity. Keep the original timestamps and a missingness indicator when the gaps matter.

5 Interpolate missing values

Interpolation estimates values between known observations. Nearest-neighbor interpolation produces steps. Linear interpolation connects neighboring values with straight lines. Cubic splines are smooth but can oscillate between points. Shape-preserving cubic interpolation limits those oscillations when monotonicity matters.

For points (x_1, y_1) and (x_2, y_2) , linear interpolation at x is

$$y(x) = y_1 + \frac{x - x_1}{x_2 - x_1}(y_2 - y_1).$$

The sample locations matter. If x is unevenly spaced, interpolating by row position gives a different answer from interpolating by the actual x values.

```
# Interpolate using the numeric index as the actual sample location.
timed = pd.Series(y.to_numpy(), index=x)
linear_by_x = timed.interpolate(method="index")
```

Limit interpolation to gaps supported by neighboring information. The following pattern fills no more than five consecutive internal missing rows. Boundary gaps and longer runs remain missing.

```
# Preserve long gaps instead of drawing unsupported straight bridges.
timed = data.set_index("time_days")["brightness"]
missing = timed.isna()
run_id = missing.ne(missing.shift()).cumsum()
run_size = missing.groupby(run_id).transform("sum")
fillable = missing & (run_size <= 5)

candidate = timed.interpolate(method="index", limit_area="inside")
timed.loc[fillable] = candidate.loc[fillable]
```

The row limit is an operational rule, not a universal scientific threshold. If sample spacing varies greatly, measure gap duration as well as row count. Store an indicator for interpolated rows when later users need to distinguish estimates from observations.

6 Shift and scale data

Normalization makes variables comparable or converts a physical measurement to a form required by a model. Each transformation answers a different question. Missing values should normally remain missing during the calculation.

6.1 Centering

Centering subtracts the mean:

$$x_{c,i} = x_i - \bar{x}.$$

The centered data have mean zero while retaining their unit and spread.

6.2 Z-scores

A z-score measures distance from the mean in standard deviations:

$$z_i = \frac{x_i - \bar{x}}{s}.$$

Specify whether s is the population standard deviation (`ddof=0`) or sample standard deviation (`ddof=1`). This course uses `ddof=0` and verifies mean 0 and standard deviation 1.

6.3 Relative and range scaling

Relative change can place a measured signal on a dimensionless scale:

$$r_i = \frac{x_i - \bar{x}}{\bar{x}}.$$

Min-max scaling maps observations to $[a, b]$:

$$x'_i = a + (b - a) \frac{x_i - x_{\min}}{x_{\max} - x_{\min}}.$$

```
# Calculate several forms without overwriting the measured values.
mean_value = x.mean()
population_sd = x.std(ddof=0)
centered = x - mean_value
z_score = (x - mean_value) / population_sd
relative = (x - mean_value) / mean_value
scaled_01 = (x - x.min()) / (x.max() - x.min())
```

Constant data have zero spread and cannot be divided by their standard deviation or range. In prediction workflows, calculate normalization parameters on the training partition, retain them, and apply the same values to validation and future observations.

7 Remove unwanted trends

Detrending separates a slowly changing baseline from the variation under study. A polynomial baseline of degree n can be fitted with `np.polyfit` and evaluated with `np.polyval`. Subtracting the fitted baseline produces the residual signal.

```
# Exclude missing values and known events from the baseline fit.
valid = values.notna() & baseline_mask
time_center = time[valid].mean()
time_scale = time[valid].std(ddof=0)
scaled_time = (time - time_center) / time_scale

coefficients = np.polyfit(scaled_time[valid], values[valid], 3)
trend = np.polyval(coefficients, scaled_time)
detrended = values - trend
```

Centering and scaling time improves numerical conditioning, particularly for higher-degree polynomials or large timestamp values. Exclude known events from the baseline fit when the events should survive in the residual. Otherwise, the baseline can bend toward them and reduce their apparent depth.

Model degree creates a bias-flexibility tradeoff. A line may leave curvature. A high-degree polynomial may follow noise or absorb the event. Compare residual structure, scientific plausibility, and performance on held-out baseline regions. The smallest training residual is insufficient evidence by itself.

8 Smooth noisy data

Smoothing reduces short-scale variation so broader patterns become easier to see. It also changes the signal. Record the method, window size, direction, and boundary behavior.

Method	Strength	Limitation
Moving mean	Reduces zero-mean random noise	Sensitive to spikes and blurs sharp changes
Moving median	Resists isolated extreme values	Can flatten small-scale continuous variation
Gaussian-weighted mean	Gives nearby values greater weight	Requires a bandwidth choice
Local regression	Follows smooth nonlinear patterns	More computation and tuning
Polynomial smoothing filter	Can preserve peaks better than a simple mean	Window and polynomial order must be compatible

```
# Sample-count windows for regularly spaced observations.
centered_mean = series.rolling(5, center=True, min_periods=1).mean()
trailing_median = series.rolling(4, min_periods=1).median()

# Reverse twice to calculate a leading four-point median.
leading_median = (
    series.iloc[::-1].rolling(4, min_periods=1).median().iloc[::-1]
)
```

A centered five-point window uses two previous values, the current value, and two future values. A trailing four-point window uses the current value and three previous values. Trailing windows support live monitoring because they do not use future data. Leading and centered windows suit retrospective analysis.

For irregular sampling, a fixed number of rows does not represent a fixed time span. Use a time-aware window. The course script includes `time_window_smooth`, which defines separate durations before and after each observation.

Larger windows suppress more noise, but they also reduce peak height, broaden transitions, and can erase short events. Compare several widths against the known time scale of the phenomenon.

9 Build a reviewable pipeline

A reliable pipeline retains raw inputs, creates named intermediate columns, and tests expected invariants. Avoid overwriting the only copy of a measurement. Export an audit table alongside the prepared observations.

```
# Check properties that must remain true after preprocessing.
assert len(final_data) == len(raw)
assert final_data["time_days"].is_monotonic_increasing
assert not (final_data["brightness_clean"] == -999).any()
assert final_data.loc[long_gap, "brightness_clean"].isna().all()
assert abs(z_score.mean()) < 1e-10
```

```
assert abs(z_score.std(ddof=0) - 1) < 1e-10

# Export without turning the DataFrame index into an extra data column.
final_data.to_csv("output/prepared_light_curve.csv", index=False)
audit.to_csv("output/quality_audit.csv", index=False)
```

9.1 Suggested output schema

Column	Meaning
time_days	Original sample location in days
brightness_clean	Sentinel and outlier handling applied; long gaps retained
relative_brightness	Centered brightness divided by its mean
trend	Cubic baseline estimated outside event intervals
detrended	Relative brightness minus the fitted trend
smoothed	Centered mean over a 0.10-day window
trailing_median	Causal median over the current and prior 0.10 days

10 Practice questions

1. Why must a sentinel code be standardized before calculating a mean or fitting a trend?
2. When does a rolling outlier detector provide a better reference than a global threshold?
3. Why can dropping missing rows make a time-series line plot misleading?
4. For irregular sample locations, why does linear interpolation by row position differ from interpolation by the measured location?
5. Give one reason to retain a long gap instead of interpolating it.
6. Which normalization keeps the original unit? Which transformations are dimensionless?
7. Why should known transit-like events be excluded from a baseline fit?
8. Which smoothing window is appropriate for live monitoring, and why?
9. Why can a 17-point moving mean obscure information that a 3-point moving mean preserves?
10. Name four assertions that should run before exporting the prepared data.

Answer guide

1. The code otherwise acts as a real measurement and can dominate summaries and fitted models.
2. It helps when the signal center changes over time and unusual values must be judged against nearby observations.
3. Removing the missing row removes the gap marker, so the plot can connect separate measured segments.
4. Row position assumes equal spacing; actual-location interpolation weights the estimate by the real distances between neighboring samples.

5. A long gap may contain unobserved structure, and a straight bridge can be mistaken for measured continuity.
6. Centering retains the unit. Z-scores, relative change, and unit-interval scaling are dimensionless.
7. Including the events can pull the baseline toward them and reduce their apparent depth in the residual.
8. A trailing window, because it uses only current and earlier observations.
9. The wider window averages across a longer interval and therefore blurs or flattens short features.
10. Examples include row count, time order, sentinel removal, long-gap preservation, expected schema, and normalization properties.

11 Quick reference

Goal	Python pattern
Locate missing values	<code>series.isna()</code>
Count missing values	<code>series.isna().sum()</code>
Standardize a sentinel	<code>series.replace(-999, np.nan)</code>
Remove incomplete rows	<code>df.dropna(subset=["value"])</code>
Fill with a median	<code>series.fillna(series.median())</code>
Interpolate by location	<code>pd.Series(y, index=x).interpolate(method="index")</code>
Centered moving mean	<code>series.rolling(5, center=True).mean()</code>
Trailing moving median	<code>series.rolling(4).median()</code>
Center values	<code>x - x.mean()</code>
Calculate z-scores	<code>(x - x.mean()) / x.std(ddof=0)</code>
Scale to [0, 1]	<code>(x - x.min()) / (x.max() - x.min())</code>
Fit a polynomial	<code>coeff = np.polyfit(x, y, degree)</code>
Evaluate a polynomial	<code>trend = np.polyval(coeff, x)</code>
Select valid rows	<code>valid = series.notna()</code>
Preserve a copy	<code>cleaned = raw.copy()</code>
Validate a requirement	<code>assert condition</code>
Export a CSV	<code>df.to_csv("prepared.csv", index=False)</code>

Completion criteria

You have completed the material when you can run both course formats, explain why each row was removed or estimated, preserve the intended long gap, compare at least two normalization and smoothing choices, and export a dataset whose quality checks pass. A preprocessing method is complete only when its assumptions and effects are visible to the next analyst.